

Social Seller — Setup Sprint 3

Inbox & Messagerie Unifiée · Prompts Claude Code · Djiguitech 2026

Objectifs du Sprint 3

US	User Story	Points	Prérequis
OAuth	Connexion OAuth canaux (WhatsApp/Facebook/TikTok)	4	Backend Sprint 2
US-09	Voir la liste des conversations dans l'inbox	5	OAuth Canaux
US-10	Ouvrir une conversation et lire les messages	5	US-09
US-11	Répondre à un message (tous canaux)	8	US-10
US-12	Messages en temps réel (Supabase Realtime)	8	US-10

Architecture Sprint 3

Nouveaux fichiers à créer :

Fichier	Description
backend/src/routes/messages.ts	POST /messages/send — envoi sortant tous canaux
backend/src/services/whatsappSendMessage.ts	Envoi message via WhatsApp Cloud API
backend/src/services/facebookSendMessage.ts	Envoi message via Facebook Send API
backend/src/services/tiktokSendMessage.ts	Envoi message via TikTok DM API
supabase/migrations/009_outbound_messages.sql	ajoute direction + delivery_status outbound
app/conversation/[id].tsx	Améliorer : affichage messages + champ réponse
app/(tabs)/inbox.tsx	Améliorer : liste conversations temps réel
lib/hooks/useRealtime.ts	Hook Supabase Realtime pour messages live

Prompt 1 — OAuth Canaux (Connexion WhatsApp/Facebook)

Ce prompt à donner à Claude Code :

Contexte projet : Social Seller SaaS multi-tenant – Expo SDK 54 + TypeScript + Expo Router + Supabase + Express sur Railway. Backend déployé sur <https://social-seller-production.up.railway.app>. Migrations 001-008 appliquées. Tâche : Implémenter le flux OAuth de connexion des canaux sociaux depuis l'app mobile. 1. Backend – route POST /connections/:platform/start (déjà dans connections.ts – vérifier et compléter) : - Créer un enregistrement oauth_states (table 008) - Retourner l'URL d'autorisation Meta (Embedded Signup pour WhatsApp/Facebook) - BACKEND_PUBLIC_URL en variable Railway pour le redirect_uri 2. Backend – route GET /oauth/facebook/callback et /oauth/whatsapp/callback (déjà dans oauth/) : - Échanger le code contre un token d'accès Meta - Chiffrer le token (tokenCrypto.ts AES-256-GCM) - Insérer dans social_connections (tenant_id, platform, access_token_enc, external_account_id) - Rediriger vers socialseller://oauth-success ou socialseller://oauth-error 3. Mobile – app/(tabs)/channels.tsx : - Bouton "Connecter" → appel

POST /connections/:platform/start → ouvrir expo-web-browser avec l'URL - Gérer le deep link retour (socialseller://oauth-success) → rafraîchir la liste - Afficher les canaux connectés (depuis social_connections via Supabase) - Bouton "Déconnecter" → DELETE /connections/:id (soft delete disconnected_at) Contraintes sécurité (CLAUDE.md) : - Valider anti-CSRF via oauth_states à chaque callback - Tous les tokens chiffrés AES-256-GCM avant écriture en base - tenant_id sur toutes les requêtes Supabase - Pas de stack trace exposée au client

Prompt 2 — Inbox US-09 & US-10 (Liste + Lecture)

Ce prompt à donner à Claude Code :

Contexte : Sprint 3 Social Seller. OAuth Canaux terminé. Les webhooks WhatsApp/Facebook alimentent les tables conversations et messages via le backend Railway. Tâche : Améliorer l'inbox et la vue conversation. 1. app/(tabs)/inbox.tsx – Liste des conversations : - Requête Supabase : conversations filtrées par tenant_id, triées par updated_at DESC - Afficher : avatar plateforme (WhatsApp/Facebook/TikTok icône colorée), nom/numéro contact, dernier message (preview 50 chars), date relative (il y a 2h) - Badge unread : count messages non lus (is_read = false) - Pull-to-refresh - État vide élégant si aucune conversation - Skeleton loading pendant le chargement 2. app/conversation/[id].tsx – Vue conversation : - Charger les messages (messages WHERE conversation_id = id, ORDER BY created_at ASC) - Bulle message entrant (gauche, fond gris) vs sortant (droite, fond indigo #6366F1) - Afficher : contenu message, heure, statut livraison (pending/sent/delivered/read) pour sortants - FlatList inversée (scroll auto vers le bas) - Header : nom contact + plateforme + bouton retour - Champ texte en bas (préparation pour US-11 – laisser placeholder) 3. Migration 009 si nécessaire : ajouter colonne is_read sur messages (bool, default false) Contraintes : tenant_id sur toutes les requêtes, RLS activé, pas de données sensibles en console.log

Prompt 3 — Réponses Sortantes US-11 (tous canaux)

Ce prompt à donner à Claude Code :

Contexte : Sprint 3 Social Seller. Inbox fonctionnel. Les marchands peuvent lire les messages. Implémenter l'envoi de réponses. Tâche : Envoi de messages sortants depuis l'inbox. 1. Backend – POST /messages/send : - Body : { conversation_id, content, tenant_id } - Vérifier JWT + tenant_id (middleware auth existant) - Récupérer la social_connection de la conversation (platform, access_token_enc, external_account_id) - Déchiffrer le token (tokenCrypto.ts) - Router vers le bon service selon platform : * WhatsApp : POST https://graph.facebook.com/v21.0/{phone_number_id}/messages * Facebook : POST https://graph.facebook.com/v21.0/me/messages * TikTok : API TikTok DM (si token disponible, sinon retourner erreur 503 propre) - Insérer le message en base (direction='outbound', delivery_status='sent') - Retourner { message_id, status } 2. Mobile – app/conversation/[id].tsx : - Champ texte + bouton Envoyer en bas de l'écran - Appel POST {EXPO_PUBLIC_API_URL}/messages/send avec JWT Authorization header - Ajouter le message localement (optimistic update) puis confirmer avec la réponse - Gérer les erreurs (network, token expiré, canal non connecté) - Désactiver le bouton pendant l'envoi (ActivityIndicator) 3. Sécurité : - Ne JAMAIS exposer le token déchiffré dans les logs - Rate limiting sur /messages/send (déjà configuré via rateLimiter.ts) - Valider que la conversation appartient bien au tenant de l'utilisateur JWT

Prompt 4 — Realtime US-12 (Messages live)

Ce prompt à donner à Claude Code :

Contexte : Sprint 3 Social Seller. Inbox + réponses fonctionnels. Ajouter le temps réel. Tâche : Supabase Realtime pour messages et conversations live. 1. lib/hooks/useRealtime.ts – Hook réutilisable : - Supabase channel subscription sur la table messages (INSERT) filtré par conversation_id - Supabase channel subscription sur la table conversations (UPDATE) filtré par

tenant_id - Cleanup automatique au unmount (removeChannel) - Retourner { newMessage, conversationUpdated } 2. app/conversation/[id].tsx : - Utiliser useRealtime({ conversationId }) pour recevoir les nouveaux messages - Append automatiquement à la FlatList sans re-fetch complet - Scroll automatique vers le bas sur nouveau message 3. app/(tabs)/inbox.tsx : - Utiliser useRealtime({ tenantId }) pour détecter les nouvelles conversations - Mettre à jour le badge unread en temps réel - Remonter les conversations avec nouveaux messages en haut de liste 4. Contraintes : - RLS sur messages/conversations assure que le client ne voit que ses données - Une seule connexion Realtime partagée (ne pas créer un channel par composant) - Gérer la reconnexion automatique (Supabase Realtime le gère nativement) - Supprimer la subscription quand le tenant se déconnecte

Checklist de validation Sprint 3

#	Test	Critère de succès
1	OAuth WhatsApp	Marchand connecte son compte WhatsApp → social_connections créé en base
2	OAuth Facebook	Marchand connecte sa page Facebook → social_connections créé en base
3	Webhook → Inbox	Message WhatsApp reçu → apparaît dans inbox sans refresh
4	Lecture conversation	Tap sur conversation → messages affichés dans l'ordre chronologique
5	Réponse WhatsApp	Marchand répond → message reçu sur WhatsApp du client
6	Réponse Facebook	Marchand répond → message reçu dans Messenger du client
7	Realtime inbox	Nouveau message entrant → badge mis à jour sans refresh
8	Realtime conversation	Nouveau message → apparaît instantanément dans la vue
9	Déconnexion canal	Marchand déconnecte → disconnected_at renseigné, canal inactif
10	Sécurité multi-tenant	Tenant A ne voit pas les conversations du Tenant B